claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a66a4c5e9735a5315.jsonl`
- SHA-256: `12d306474d7e7fd13534e33e0f482b779b7c5bf921f8f45833cb7318a0f19985`
- Source modified: `2026-06-25T16:57:35+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T16:57:09.964Z)

```
Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is
checked out; read the real code with git diff origin/master...HEAD + Read the files). Be
skeptical ? default to isReal=false / adjustedSeverity=INVALID if the finding is wrong, already
handled by the code, or a non-issue. Do NOT re-run CI.

Finding [NIT] Inaccurate comment: prod does not use _safe_delete in the sweep loop
Location: tests/test_provider_gemini.py:160
Detail: The comment 'First delete raises, second succeeds ? _safe_delete swallows it.' is wrong.
cleanup_orphaned_files calls `self.client.files.delete(name=f.name)` directly inside its own
`try/except` (gemini.py:78-81); it does not route through `_safe_delete` (gemini.py:37). The
test's assertion (deleted==1) is correct, but the rationale misattributes the swallowing
mechanism, which could mislead a future maintainer.
Suggested fix: Change the comment to '? the per-file except in cleanup_orphaned_files swallows
it.'

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the
actual code.
```

2. assistant (2026-06-25T16:57:12.492Z)

```
I'll verify this finding by reading the actual code on the PR branch.
```

3. user (2026-06-25T16:57:13.608Z)

```
 backend/main.py               |  25 ++++++++++
 backend/providers/gemini.py   |  43 ++++++++++++++++++
 tests/test_provider_gemini.py | 104 ++++++++++++++++++++++++++++++++++++++++++++
 3 files changed, 172 insertions(+)
```

4. assistant (2026-06-25T16:57:15.454Z)

```
Let me read the actual code in the relevant files.
```

5. user (2026-06-25T16:57:16.600Z)

```
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
 import logging
 import os
 import time
```

```
+from datetime import datetime, timedelta, timezone
 from typing import Tuple

 from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
         except Exception:
             pass

+    def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+        """Sweep and delete video files orphaned on the Gemini File API by a previous
+        crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+        leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+        that reach a ``finally``/except).
+
+        Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+        creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+        Never raises; returns the count deleted. Intended to run once at startup.
+
+        Args:
+            grace_sec: only delete files older than this (default 1h). Protects a file a
+                live worker may still be processing.
+        """
+        cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+        deleted = 0
+        try:
+            files = list(self.client.files.list())
+        except Exception as e:
+            logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+            return 0
+        for f in files:
+            try:
+                mime = getattr(f, "mime_type", "") or ""
+                if not mime.startswith("video/"):
+                    continue
+                # f.create_time is an RFC3339 string on the File API.
+                ct = getattr(f, "create_time", None)
+                if ct is None:
+                    # No timestamp ? can't prove it's stale; leave it (safe default).
+                    continue
+                created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+                if created > cutoff:
+                    continue  # too recent ? a live worker may own it
+                self.client.files.delete(name=f.name)
+                deleted += 1
+            except Exception as e:
+                logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+        if deleted:
+            logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+        return deleted
+
     def analyze_video(self, video_path: str, prompt: str,
                       chunk_duration: float = 0.0,
                       label: str = "") -> Tuple[list, dict]:
```

6. user (2026-06-25T16:57:17.416Z)

```
1       import json
2       import logging
3       import os
4       import time
5       from datetime import datetime, timedelta, timezone
6       from typing import Tuple
7
8       from google import genai
```

```
    from google.genai import types

    from backend.infrastructure.execution_policy import (
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
        run_bounded,
    )
    from backend.providers.base import AIVideoProvider

    logger = logging.getLogger(__name__)

    # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
    # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
    MAX_UPLOAD_MB = 500

    class GeminiProvider(AIVideoProvider):
        def __init__(self, api_key: str, model_name: str):
            super().__init__(api_key, model_name)
            # Initialize client lazily to avoid connection/auth issues during class
instantiation
            self._client = None

        @property
        def client(self):
            if self._client is None:
                self._client = genai.Client(api_key=self.api_key)
            return self._client

        def _safe_delete(self, video_file) -> None:
            """Best-effort cleanup of an uploaded File (never raises)."""
            try:
                self.client.files.delete(name=video_file.name)
            except Exception:
                pass

        def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
            """Sweep and delete video files orphaned on the Gemini File API by a previous
            crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
            leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
            that reach a ``finally``/except).

            Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
whose
            creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
upload).
            Never raises; returns the count deleted. Intended to run once at startup.

            Args:
                grace_sec: only delete files older than this (default 1h). Protects a file a
                    live worker may still be processing.
            """
            cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
            deleted = 0
            try:
                files = list(self.client.files.list())
            except Exception as e:
                logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
                return 0
            for f in files:
                try:
                    mime = getattr(f, "mime_type", "") or ""
                    if not mime.startswith("video/"):
```

```
                        continue
                    # f.create_time is an RFC3339 string on the File API.
                    ct = getattr(f, "create_time", None)
                    if ct is None:
                        # No timestamp ? can't prove it's stale; leave it (safe default).
                        continue
                    created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
                    if created > cutoff:
                        continue  # too recent ? a live worker may own it
                    self.client.files.delete(name=f.name)
                    deleted += 1
                except Exception as e:
                    logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
            if deleted:
                logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
            return deleted

        def analyze_video(self, video_path: str, prompt: str,
                          chunk_duration: float = 0.0,
                          label: str = "") -> Tuple[list, dict]:
            log_prefix = f"Gemini {label}" if label else "Gemini"

            metrics: dict = {
                "upload_time": 0.0,
                "process_time": 0.0,
                "gen_time": 0.0,
            }

            # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
            try:
                size_mb = os.path.getsize(video_path) / (1024 * 1024)
            except OSError:
                size_mb = 0.0
            if size_mb > MAX_UPLOAD_MB:
                logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
                             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
                             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
                metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
                metrics["oversize_skipped"] = True
                return [], metrics

            # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
            logger.info(f"[{log_prefix}] Uploading {video_path}...")
            t_upload_start = time.time()
            video_file = None
            for attempt in range(3):
                try:
                    video_file = self.client.files.upload(file=video_path)
                    metrics["upload_time"] = time.time() - t_upload_start
                    logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
                    break
                except Exception as e:
                    logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
                    if attempt < 2:
                        time.sleep(2 * (attempt + 1))
            if video_file is None:
                logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
                metrics["error"] = "upload failed after 3 attempts"
```

```
127                 return [], metrics
128
129             # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
130             # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131             t_process_start = time.time()
132             process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0)  #
20 min cap
133
134             def _processed():
135                 nonlocal video_file
136                 video_file = self.client.files.get(name=video_file.name)
137                 return None if video_file.state.name == "PROCESSING" else video_file
138
139             try:
140                 video_file = poll_until(_processed, process_policy,
141                                         label=f"{log_prefix} processing", logger=logger)
142             except PolicyTimeout:
143                 logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
144                 self._safe_delete(video_file)
145                 metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
146                 return [], metrics
147             except Exception as e:
148                 logger.error(f"[{log_prefix}] Status check failed: {e}")
149                 self._safe_delete(video_file)
150                 metrics["error"] = f"status check failed: {e}"
151                 return [], metrics
152             if video_file.state.name == "FAILED":
153                 logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
154                 self._safe_delete(video_file)
155                 metrics["error"] = f"video processing failed: {video_file.error.message}"
156                 return [], metrics
157
158             metrics["process_time"] = time.time() - t_process_start
159             logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
160
161             # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
162             # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
163             # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
164             logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
165             t_gen_start = time.time()
166             gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167             raw_text = None
168
169             def _generate():
170                 return self.client.models.generate_content(
171                     model=self.model_name,
172                     contents=[video_file, prompt],
173                     config=types.GenerateContentConfig(
174                         response_mime_type="application/json", temperature=0.0),
175                 )
176
177             try:
178                 response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
```

```
179                 raw_text = response.text
180                 metrics["gen_time"] = time.time() - t_gen_start
181                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182             except Exception as e:                  # PolicyTimeout (hang) or the last 503 ?
both end here
183                 logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184                 metrics["error"] = f"generate failed after retries: {e}"
185                 metrics["generate_failed"] = True
186             self._safe_delete(video_file)
187
188             if not raw_text:
189                 return [], metrics
190
191             # 4. Parse JSON
192             try:
193                 segments = json.loads(raw_text)
194                 if not isinstance(segments, list):
195                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
196                     metrics["error"] = "response is not a JSON array"
197                     return [], metrics
198                 return segments, metrics
199             except Exception as e:
200                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
201                 metrics["error"] = f"JSON parse failed: {e}"
202                 return [], metrics
203
```

7. user (2026-06-25T16:57:17.667Z)

```
1       import logging
2       from unittest.mock import MagicMock, patch
3
4       from backend.providers.gemini import GeminiProvider
5
6
7       def test_gemini_provider_generate_failure_is_flagged():
8           """A generate() give-up (per-attempt timeout / 503 burst after retries) must set
9           metrics['generate_failed'] so a FAILED chunk isn't silently mistaken for 'no
rallies'
10          (the silent-partial-result guard the segmenter surfaces loudly)."""
11          with patch("backend.providers.gemini.genai") as mock_genai, \
12               patch("backend.providers.gemini.run_bounded", side_effect=RuntimeError("503
exhausted")):
13              client_mock = MagicMock()
14              mock_genai.Client.return_value = client_mock
15              file_mock = MagicMock()
16              file_mock.state.name = "ACTIVE"
17              file_mock.name = "dummy_file"
18              client_mock.files.upload.return_value = file_mock
19              client_mock.files.get.return_value = file_mock
20
21              provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
22              segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
23
24              assert segments == []                       # no analysis for this chunk
25              assert metrics.get("generate_failed") is True   # ...but the failure is FLAGGED,
not silent
26
27
28      def test_gemini_provider_success():
29          # Mock GenAI client
30          with patch("backend.providers.gemini.genai") as mock_genai:
```

```
            client_mock = MagicMock()
            mock_genai.Client.return_value = client_mock

            # Mock file upload
            file_mock = MagicMock()
            file_mock.uri = "gs://dummy"
            file_mock.name = "dummy_file"
            file_mock.state.name = "ACTIVE"

            client_mock.files.upload.return_value = file_mock
            client_mock.files.get.return_value = file_mock

            # Mock generate_content response
            response_mock = MagicMock()
            response_mock.text = '[{"start_time": 1.0, "end_time": 2.0}]'
            client_mock.models.generate_content.return_value = response_mock

            provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
            segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)

            assert len(segments) == 1
            assert segments[0]["start_time"] == 1.0
            assert metrics["upload_time"] >= 0
            assert metrics["process_time"] >= 0
            assert metrics["gen_time"] >= 0

            # Ensure delete is called
            client_mock.files.delete.assert_called_with(name="dummy_file")

    def test_gemini_provider_invalid_json():
        with patch("backend.providers.gemini.genai") as mock_genai:
            client_mock = MagicMock()
            mock_genai.Client.return_value = client_mock
            file_mock = MagicMock()
            file_mock.state.name = "ACTIVE"
            client_mock.files.upload.return_value = file_mock
            client_mock.files.get.return_value = file_mock

            response_mock = MagicMock()
            response_mock.text = '{"not_a_list": true}'
            client_mock.models.generate_content.return_value = response_mock

            provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
            segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
            assert len(segments) == 0

    def test_gemini_provider_rejects_oversize_before_upload(monkeypatch, caplog):
        with patch("backend.providers.gemini.genai") as mock_genai:
            client_mock = MagicMock()
            mock_genai.Client.return_value = client_mock
            # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
            monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
                                lambda p: 600 * 1024 * 1024)
            provider = GeminiProvider(api_key="k", model_name="gemini-flash-latest")
            with caplog.at_level(logging.ERROR, logger="backend.providers.gemini"):
                segments, metrics = provider.analyze_video("big.mp4", "p")
            assert segments == []
            client_mock.files.upload.assert_not_called()
            # The refusal must be LOUD and machine-visible: callers aggregate the metrics
            # flag into the per-video report (oversize_clip_skipped rule).
            assert metrics["oversize_skipped"] is True
            assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
```

```
93
94
95      def test_gemini_provider_upload_fail():
96          with patch("backend.providers.gemini.genai") as mock_genai:
97              client_mock = MagicMock()
98              mock_genai.Client.return_value = client_mock
99              client_mock.files.upload.side_effect = Exception("Upload err")
100
101             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
102             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
103             assert len(segments) == 0
104
105
106
107     # --- O1/P11: Gemini remote-file GC (startup orphan sweep) ---
108
109     def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
110         f = MagicMock()
111         f.name = name
112         f.mime_type = mime
113         f.create_time = create_time_iso
114         f.state = MagicMock()
115         f.state.name = state
116         return f
117
118
119     def test_cleanup_orphaned_files_deletes_stale_videos_only():
120         """Deletes video files older than the grace window; leaves non-videos and fresh
files."""
121         from datetime import datetime, timedelta, timezone
122         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
123         fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
124         files = [
125             _file_mock("old-vid", "video/mp4", stale),       # stale video -> deleted
126             _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
127             _file_mock("old-img", "image/png", stale),        # non-video -> kept
128             _file_mock("notts", "video/mp4", None),           # no timestamp -> kept (safe)
129         ]
130         with patch("backend.providers.gemini.genai"):
131             provider = GeminiProvider(api_key="k", model_name="m")
132             provider._client = MagicMock()
133             provider._client.files.list.return_value = iter(files)
134             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
135         assert deleted == 1
136         provider._client.files.delete.assert_called_once_with(name="old-vid")
137
138
139     def test_cleanup_orphaned_files_list_failure_is_swallowed():
140         """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
141         with patch("backend.providers.gemini.genai"):
142             provider = GeminiProvider(api_key="k", model_name="m")
143             provider._client = MagicMock()
144             provider._client.files.list.side_effect = RuntimeError("network")
145             assert provider.cleanup_orphaned_files() == 0
146
147
148     def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
149         """A delete() failure on one file must not abort the sweep."""
150         from datetime import datetime, timedelta, timezone
151         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
152         files = [
153             _file_mock("a", "video/mp4", stale),
154             _file_mock("b", "video/mp4", stale),
```

```
155         ]
156         with patch("backend.providers.gemini.genai"):
157             provider = GeminiProvider(api_key="k", model_name="m")
158             provider._client = MagicMock()
159             provider._client.files.list.return_value = iter(files)
160             # First delete raises, second succeeds ? _safe_delete swallows it.
161             provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
163         assert deleted == 1  # only the second file counted
164
165
166
167     # --- O1/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
168
169     def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
170         """A non-Gemini provider must short-circuit (no client built, no sweep)."""
171         import backend.main as main_mod
172         cfg = MagicMock()
173         cfg.ai_provider = "openai"
174         called = []
175         monkeypatch.setattr(main_mod.provider_registry, "get",
176                             lambda *a, **k: called.append(k) or MagicMock())
177         monkeypatch.setenv("GEMINI_API_KEY", "k")
178         main_mod._maybe_sweep_gemini_orphans(cfg)
179         assert called == []  # provider never instantiated
180
181
182     def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
183         """No GEMINI_API_KEY -> no sweep (can't auth)."""
184         import backend.main as main_mod
185         cfg = MagicMock()
186         cfg.ai_provider = "gemini"
187         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
188         called = []
189         monkeypatch.setattr(main_mod.provider_registry, "get",
190                             lambda *a, **k: called.append(k) or MagicMock())
191         main_mod._maybe_sweep_gemini_orphans(cfg)
192         assert called == []
193
194
195     def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196         """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
197         import backend.main as main_mod
198         cfg = MagicMock()
199         cfg.ai_provider = "gemini"
200         cfg.ai_model = "gemini-2.5-flash"
201         provider = MagicMock()
202         provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
203         monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
204         monkeypatch.setenv("GEMINI_API_KEY", "k")
205         # Must not raise.
206         main_mod._maybe_sweep_gemini_orphans(cfg)
207         provider.cleanup_orphaned_files.assert_called_once()
208
```

8. assistant (2026-06-25T16:57:27.499Z)

I have all the information I need. Let me verify the finding precisely.

The finding is about line 160 of `tests/test_provider_gemini.py`:

```
# First delete raises, second succeeds ? _safe_delete swallows it.
```

Looking at the actual code:

1. **`cleanup_orphaned_files` (gemini.py:44-84)**: Inside its sweep loop at line 78, it calls `self.client.files.delete(name=f.name)` **directly**, NOT through `_safe_delete`. The exception that a failing delete throws is caught by the per-file `try/except` at lines 66-81 (`except Exception as e: logger.debug(...)` at line 80).

2. **`_safe_delete` (gemini.py:37-42)**: A separate method that is only called in `analyze_video` (lines 144, 149, 154, 186), never in `cleanup_orphaned_files`.

3. **The test (`test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed`, lines 148-163)**: It exercises `cleanup_orphaned_files`, where the first `delete` raises `RuntimeError("boom")` and the second succeeds. The swallowing mechanism that makes `deleted == 1` true is the **per-file `except Exception` block inside the sweep loop** (line 80), NOT `_safe_delete`.

The finding is factually correct: the comment misattributes the swallowing to `_safe_delete`, when in reality `cleanup_orphaned_files` never routes through `_safe_delete` ? it catches the exception in its own inline `try/except`. This is a genuine inaccuracy in a test comment that could mislead a future maintainer into thinking the two code paths share the same cleanup helper. The assertion itself is correct. Severity NIT is appropriate ? it's a comment-only inaccuracy in a test, with no functional impact.

9. user (2026-06-25T16:57:31.413Z)

Structured output provided successfully

10. assistant (2026-06-25T16:57:35.185Z)

Verified. The finding is real and correctly classified as NIT.

The test comment at `tests/test_provider_gemini.py:160` misattributes the exception-swallowing to `_safe_delete`, but `cleanup_orphaned_files` (`backend/providers/gemini.py:44-84`) calls `self.client.files.delete()` directly at line 78 inside its own per-file `try/except` (the `except` at line 80 is what swallows the error). `_safe_delete` (lines 37-42) is only used in `analyze_video`. The assertion is correct; only the comment's rationale is wrong, so it's a harmless cosmetic inaccuracy.